

Explicacion logica de dos CRUD - Proyecto Cruz Negra

Que significa CRUD

CRUD representa cuatro operaciones basicas:

C - Create: crear un registro.

R - Read: consultar o listar registros.

U - Update: actualizar un registro.

D - Delete: eliminar un registro.

Flujo general en Laravel

Vista Blade -> Ruta -> Controller -> Validacion -> Model -> Base de datos

La vista muestra el formulario o la tabla. La ruta recibe la peticion

y llama al metodo correspondiente del Controller. El Controller valida

los datos y utiliza el Model Eloquent para leer o modificar la base.

CRUD 1: Tareas - CRUD con dependencias

Archivos principales

app/Http/Controllers/TareaController.php

app/Models/Tarea.php

resources/views/tareas/index.blade.php

routes/web.php

Por que tiene dependencias

Una tarea se relaciona con otras entidades mediante claves foraneas:

proyecto_id -> proyecto al que pertenece.

sprint_id -> sprint o etapa de trabajo.

asignado_a -> usuario responsable.

solicitud_cambio_id -> solicitud de cambio opcional.

El modelo Tarea define relaciones belongsTo con Proyecto, Sprint, User y SolicitudCambio. Por eso, para completar una tarea se necesitan datos que provienen de otras tablas.

Logica del CRUD de Tareas

1. Listar

TareaController::index obtiene las tareas visibles para el usuario.

Carga proyecto y usuario asignado con with(), aplica busqueda y filtros, ordena, pagina y envia los datos a tareas.index.

2. Crear

TareaController::store valida titulo, estado, prioridad y fecha.

Luego ejecuta Tarea::create(\$data) y redirige al listado.

3. Editar

El boton de editar envia data-url y data-valores al modal.

crud-modal.js completa el formulario y cambia su action.

TareaController::update valida y ejecuta \$tarea->update(\$data).

4. Eliminar

TareaController::destroy ejecuta \$tarea->delete() y vuelve al listado.

CRUD 2: Usuarios - entidad principal con relaciones

Archivos principales

app/Http/Controllers/UserController.php

app/Models/User.php

resources/views/users/index.blade.php

routes/web.php

1. Listar

`UserController::index` busca usuarios, carga el cliente asociado, aplica busqueda por nombre, apellido o email y pagina los resultados. Tambien obtiene los roles y clientes para los formularios.

2. Crear

`UserController::store` valida nombre, email, estado, password y rol. La regla unique evita emails repetidos y exists verifica que el rol exista. La password se protege con `Hash::make` antes de guardarse. Finalmente `syncRoles` asigna el rol seleccionado.

Logica del CRUD de Usuarios

3. Editar

El modal recibe los valores actuales mediante data-valores.

UserController::update valida los cambios y guarda el usuario.

La password es opcional: si queda vacia, conserva la anterior.

4. Cambiar rol

updateRoles valida el rol y usa syncRoles para reemplazar el rol actual.

5. Eliminar

destroy aplica reglas de negocio antes de borrar:

- No permite eliminar la propia cuenta.
- No permite eliminar al unico usuario con rol Jefe.

Luego ejecuta \$user->delete().

El usuario se relaciona con Cliente y con el sistema de roles.

Por eso tiene dependencias, aunque sus datos principales pertenecen a la tabla users.

Guion breve para la exposicion

En nuestro proyecto implementamos dos CRUD usando Laravel.

El CRUD de usuarios permite crear, listar, editar, cambiar roles y eliminar cuentas. El Controller recibe las peticiones, valida los datos, utiliza el modelo User y devuelve una vista o redireccion.

El CRUD de tareas tiene una logica similar, pero agrega dependencias con otras entidades. Una tarea pertenece a un proyecto, puede estar asignada a un usuario, pertenecer a un sprint y relacionarse con una solicitud de cambio. Por eso utiliza relaciones belongsTo.

La diferencia principal es que el CRUD de tareas tiene dependencias fuertes con otros modelos, mientras que el CRUD de usuarios administra principalmente su propia entidad y agrega relaciones con roles y clientes.